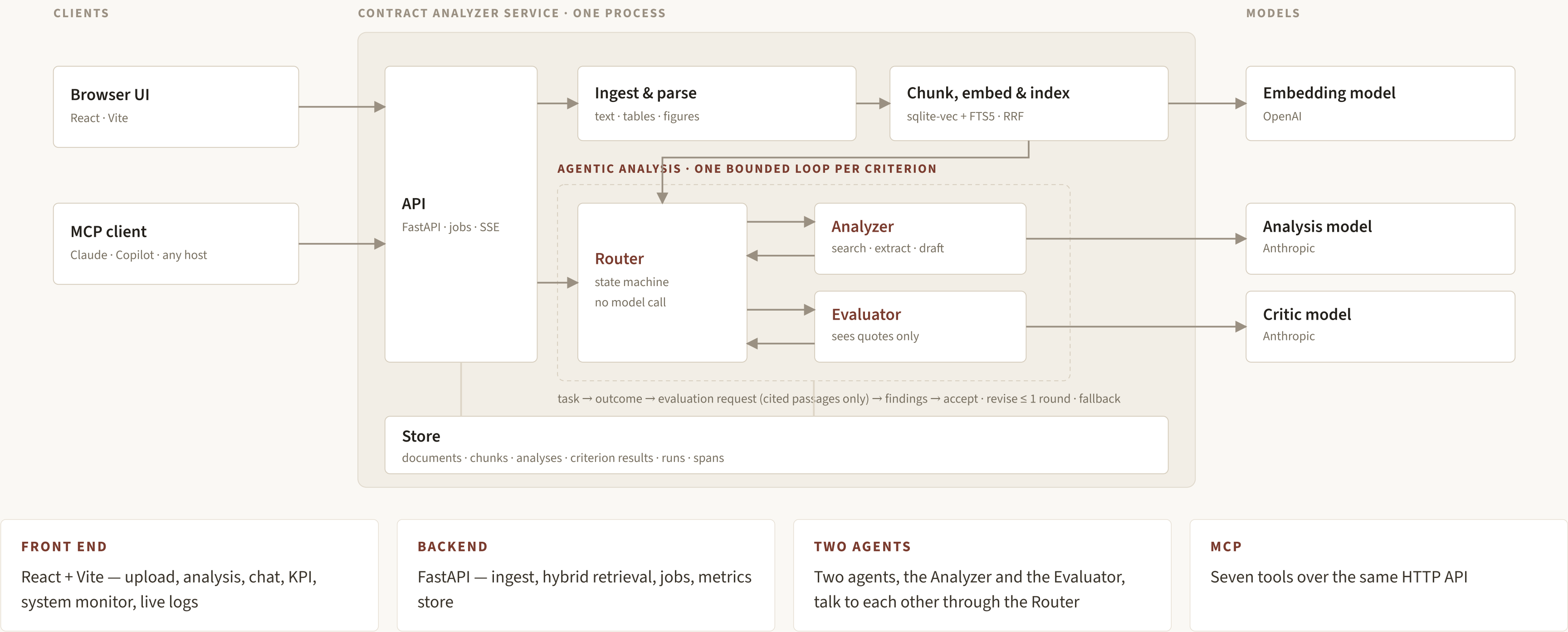
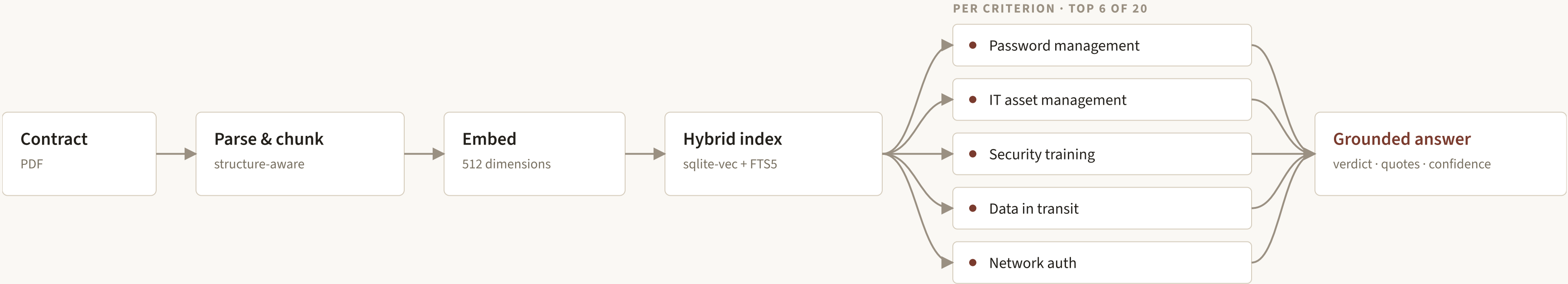

Contract Analyzer

High-level architecture



The RAG pipeline

Five criteria, five independent retrievals over the same contract — each scoped to its own question, so a password clause never crowds out an encryption one.



A PDF is a display list, not a document

It records where to paint glyphs — not words, not reading order, not that a grid of strings is a table. So the parser measures the document before it reads it, and hands the chunker typed elements rather than a wall of text.

THE RULE EVERY HEURISTIC IS HELD TO

A decision may only use evidence the document supplies about itself.

01 Measure the document

02 Read each page in claim order

03 Rebuild the structure

What comes out: typed elements

sample · 21 pages

ELEMENT	WHAT IT CARRIES BEYOND THE TEXT	COUNT
heading	Pins where a section starts, so the spine follows the title rather than the page top	51
paragraph	One clause — rejoined from rendered lines, across page breaks, never across a clause boundary	79
table	Markdown and rows, a quality grade, a caption; 10 elements span a page break	34
figure caption equation	Also modelled and handled — assets on disk, captions claimed with their owner, math by font coverage, running heads kept aside rather than discarded. This contract has none of them.	

EVERY ELEMENT, WHATEVER ITS TYPE

text · page_index · page_label · page_span · section_path · order · bbox

Analysis: Two agents, One Deterministic Orchestrator

The Analyzer cannot see its own errors — a verbatim quote that supports nothing passes every structural rule. So the Evaluator re-derives the link from scratch.

Analyzer

model call • analysis_model

DECIDES

What to search and what to extract; drafts the result and fixes its own structure.

Evaluator

model call • the critic

DECIDES

Deterministic pre-checks first, then whether each quote actually carries the claim it was cited for.

SEES

The claims and the cited passages. Not the reasoning — a critic shown the reasoning inherits the error that reasoning made.

ROUTER • NOT AN AI AGENT

no model call

```
outcome = Analyzer(criterion)
findings = Evaluator(claims + cited passages)
decision = decide(findings, round) # a table, not a model
  accept • fallback → ship      •  revise(mode) → Analyzer.revise(...)
```


Chat: Free-form questions, same evidence

Anything beyond the five criteria runs the same tool loop over the same index — and answers only from what it retrieved.

IT REFUSES RATHER THAN GUESSES

If the loop retrieves nothing, chat stops there and says so. A second request over an empty ledger is an invitation to answer from general knowledge, which is the one thing a compliance tool must not do.

CITATIONS ARE OFFSETS, NOT FOOTNOTES

Each retrieved passage is sent as its own document block with citations on, so every claim comes back as a character range into a named section — the UI highlights the exact span rather than linking to a page.

SAME MACHINERY, SMALLER BUDGET

Retrieval	the same hybrid index
Tool calls	4 · analysis gets 8
Effort	low · analysis is medium
Answer	streamed, cited

THREE CONTROLS, ABOVE THE BOX

Model claude-sonnet-5 · ...	the allowlist the API enforces
Retrieval hybrid · vector · keyword	how the contract is searched
Depth shallow · medium · deep	how much evidence is sent

Applies to the next question. Depth hides top_k on purpose — medium is the configured default; half and double are a labelled placeholder.

One core, two front doors

MCP server has no authentication, that is because this is a Demo, in production OAuth compatible with organization central OAuth, and a role based access management system and API usage monitor needed.

REST API

OPENAPI

- POST

/api/documents

Upload a PDF — parse, chunk, embed, index
- POST

/api/analyses

Start the five-criterion run; returns at once
- GET

/api/analyses/{analysis_id}/events

SSE phases: retrieving · evaluating · revising
- GET

/api/analyses/{analysis_id}

The validated JSON, per the output schema
- GET

/api/metrics/summary

Feeds the KPI dashboard
- GET

/api/metrics/runs/{run_id}/spans

The trace behind any single answer

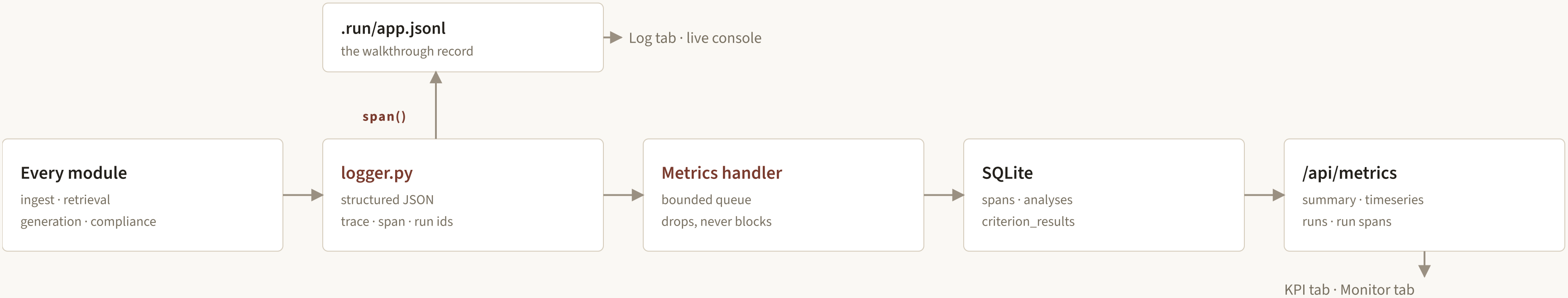
MCP server

7 TOOLS

- get_started
- upload_contract
- list_contracts
- list_criteria
- analyze_compliance
- get_analysis
- search_contract

How metrics get collected

Modules only ever call `span()`. A logging handler turns every `span().end` into a row — so no module knows the metrics store exists.



ONE RECORD, TWO READERS

The log file and the table are the same records, so the walkthrough and the aggregate cannot disagree.

KPI: numbers, and why each one

The dashboard actually shows, in its own three bands. The time-series charts plot these same numbers over the window; they are not extra metrics.

METRIC	THRESHOLD (ARBITRARY)	WHAT IT IS FOR
NOW		
Runs	—	Documents and criteria in the window — the denominator under every rate below
Failure rate	≤ 2%	Failed plus interrupted. Needs-review is quality and is deliberately not in this
p95 job duration	≤ 120 s	The tail is what breaks a demo, which is why this is p95 and not the mean
Total spend	\$50 / day	A breach pauses new runs rather than letting an overrun go quiet
p95 job cost	≤ 2× p50	A ratio, not a dollar cap: one run at twice the median is one that went wild
ANSWER QUALITY		
Quote verification	≥ 99%	Quotes found verbatim in the passage they cite — the hallucination check
Needs review	≤ 10%	Flagged for a human; rising means harder contracts or worse retrieval
WHERE THE MONEY WENT		
Cost by model	—	Which model billed, read off agent.call spans
Analysis vs chat	—	Chat as a share of billed spend — a split of the money, not of the surface

Every run on the dashboard carries the trace id it was made under, so any number here can be followed into the log — that link is the reason the runs table exists at all.

Monitoring and logging

KPIs say whether the answer is good. These say whether the box is healthy, and let you track issues.

What we monitor

30 m • 1 h

API cost in KPI, other infra cost should be included in future.

UPSTREAM exhausted ≤ 1%

Retries per 100 calls to Anthropic and OpenAI, how often the retry budget runs out, and the HTTP status or error type behind it.

STAGES ≤ 5% • n ≥ 10

Error rate and error count for the worst named pipeline stage, read off the same spans the KPIs come from.

HOST tight over 90%

Memory and disk as a share of the machine.

How logging works

logger.py

One module is the only logging surface.

ONE CONTEXT

A request stamps **trace and span** into context vars once. Everything under that request inherits those IDs automatically.

TWO SINKS, ONE RECORD

A compact line to the console, JSON to `.run/app.jsonl`. The Log tab streams those records over SSE to front end.

THE SPAN FAMILIES

```
ingest.file • parse • chunk • embed • write
retrieve • agent.call • agent.tool • chat
router.criterion • analysis.criterion • evaluator.precheck
upstream.call • upstream.retry • upstream.failed
```

Trade Offs

CHOSE	OVER	WHY
SQLite, one file	Postgres + pgvector	The panel runs it with no services. Swapping the store is not a rewrite.
A deterministic Router	An LLM router	Every input to it is already a structured judgement. A model re-reading that JSON adds latency, cost and a failure mode.
No agent framework	A graph library	The loop is thirty lines. A framework adds a second retry layer against the one-transport rule, and hides the log story. Frameworks such as lang-chain add unnecessary complexity for this project
RRF over ranks	Tuned hybrid weights	An L2 distance and a BM25 score are not on the same scale, and no weighting of them is defensible without a labelled set to tune on.

Limitations

**No labelled test data to
measure accuracy,
precision or recall**

**Uncalibrated confidence
calculation**

Untuned prompts and loops

Live demo

RUNNING AT

WEB APP `demo.amirrezaslh.com`

MCP `demo.amirrezaslh.com/mcp`

RUNNING ORDER

01 Upload a contract

02 Run an analysis, or chat with the document

03 Trace performance through the KPI, Monitor and Log tabs